

WEEK 3: SELLER AND INVENTORY MANAGEMENT SYSTEM

Scenario:

The e-commerce company sells products from different sellers. The system stores seller details, product information, and stock levels to make sure products are available.

1. Create Seller and Inventory Tables

```
SQL> CREATE TABLE Seller (  
2     Seller_ID VARCHAR(7),  
3     Seller_Name VARCHAR(25),  
4     Email VARCHAR(30),  
5     Phone NUMBER(10),  
6     Address VARCHAR(45)  
7 );
```

Table created.

```
SQL> CREATE TABLE Inventory (  
2     Inventory_ID VARCHAR(7),  
3     Product_ID VARCHAR(7),  
4     Seller_ID VARCHAR(7),  
5     Stock_Quantity NUMBER,  
6     Stock_Status VARCHAR(11),  
7     Last_Updated DATE  
8 );
```

Table created.

2. Establish Relationships

Seller and Product

- A seller can provide many products.
- A product can be provided by one or more sellers.
- **Relationship:** Seller → Product is **one-to-many**.

Product and Inventory

- Each product has its own inventory details.
- Inventory keeps track of the quantity available.
- **Relationship:** Product → Inventory is **one-to-one**.

Primary Keys:

- Seller_ID is the primary key for the Seller table.
- Inventory_ID is the primary key for the Inventory table.

Primary Key added for Both Tables:

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
Table altered.
```

Foreign Key for Inventory Table:

```
SQL> ALTER TABLE Inventory ADD
  2  CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_I
D);
Table altered.

SQL> ALTER TABLE Inventory ADD
  2  CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID
);
Table altered.
```

Constraints to make Unique Email ID:

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);
Table altered.
```

To make Email NOT NULL

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity >= 0)
;
Table altered.
```

Insert Values to Check the Stock Quantity is Positive

```
SQL> INSERT INTO Seller VALUES('&SellID', '&SellerName', '&email', &phone, '
&Address');
Enter value for sellid: S00001
Enter value for sellername: Vani Enterprises
Enter value for email: mdvanienter@gmail.com
Enter value for phone: 8765432109
Enter value for address: 1, Anna Nagar, Chennai
old 1: INSERT INTO Seller VALUES('&SellID', '&SellerName', '&email', &phon
e, '&Address')
new 1: INSERT INTO Seller VALUES('S00001', 'Vani Enterprises', 'mdvaniente
r@gmail.com', 8765432109, '1, Anna Nagar, Chennai')

1 row created.
```

```
SQL> SELECT * FROM Seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S00001	Vani Enterprises 1, Anna Nagar, Chennai	mdvanienter@gmail.com	8765432109
S00004	DSVV Manufaturers 23, Thillai Nagar, Trichy	dsvvmanuf142@gmail.com	7824003009
S00003	AtoZ 90, George Town, Chennai	atozmarket@gmail.com	7890654312

SELLER_	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S00002	SD SET DEFINE OFF; 62, Thirumal Nagar, Thanjavur	sellerD827@gmail.com	9876543012

Insert Values to Inventory Table

```
SQL> INSERT INTO Product (Product_ID, Product_Name) VALUES ('P100002', 'Sample Product 2');
```

```
1 row created.
```

```
SQL> INSERT INTO Product (Product_ID, Product_Name) VALUES ('P200002', 'Sample Product 3');
```

```
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('Inv0003', 'P100002', 'S00003', 4, 'Unavailable', TO_DATE('2026-09-15', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('Inv0004', 'P200002', 'S00004', 3, 'Available', TO_DATE('2026-09-14', 'YYYY-MM-DD'));
```

```
1 row created.
```

```
SQL> SELECT * FROM Inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
-----	-----	-----	-----	-----	-----
Inv0001	P100001	S00001	4	Available	27-SEP-26
Inv0002	P100001	S00002	3	Available	13-AUG-26
Inv0003	P100002	S00003	4	Unavailable	15-SEP-26
Inv0004	P200002	S00004	3	Available	14-SEP-26

Report 1: Seller-wise Product Report

Objective

This report aggregates inventory data to evaluate each seller's contribution by displaying their total product varieties supplied and total current stock quantities in the warehouse.

Required Display Fields

1. **Seller Name:** The full registered name of the vendor/seller.
2. **Number of Products Supplied:** The total count of distinct products linked to the seller.
3. **Available Stock:** The total sum of units available across all products for that seller.

```

SQL> SELECT s.Seller_Name,
2         COUNT(i.Product_ID) AS Number_of_Products,
3         SUM(i.Stock_Quantity) AS Available_Stock
4 FROM Seller s
5 JOIN Inventory i ON s.Seller_ID = i.Seller_ID
6 WHERE i.Stock_Status = 'Available'
7 GROUP BY s.Seller_Name;

```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
SD SET DEFINE OFF;	1	3
DSVV Manufaturers	1	3
Vani Enterprises	1	4

Use Joins to get details from Seller and Inventory Table.

Report 2: Stock Availability Report

Objective

This report provides clear visibility into current warehouse stock levels and availability statuses across all product items. It allows inventory managers to monitor stock quantities in real time and quickly identify out-of-stock or unavailable products needing reorders.

Display Requirements

- **Product name:** Name of the catalog product.
- **Stock quantity:** Number of units currently held in inventory.
- **Stock status:** Operational stock condition (e.g., *Available*, *Unavailable*).

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status
2 FROM Product p
3 JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

```
PRODUCT_NAME
```

```
-----
```

```
STOCK_QUANTITY STOCK_STATU
```

```
-----
```

```
Sample Product
4 Available
```

```
Sample Product
3 Available
```

```
Sample Product 2
4 Unavailable
```

```
PRODUCT_NAME
```

```
-----
```

```
STOCK_QUANTITY STOCK_STATU
```

```
-----
```

```
Sample Product 3
3 Available
```

Report 3: Inventory Analysis

Objective

This report provides high-level summary metrics for overall inventory health. It quantifies active, ready-to-sell stock entries versus entries that are completely out of stock, offering warehouse managers immediate executive visibility into stock fulfillability.

Analysis Tasks & SQL Queries

1. Total Products Available

```
SQL> SELECT COUNT(*) AS Total_Available_Products
2 FROM Inventory
3 WHERE Stock_Status = 'Available';
```

```
TOTAL_AVAILABLE_PRODUCTS
```

```
-----
```

```
3
```

2. Products out of stock

```
SQL> SELECT COUNT(*) AS OutOfStock_Products
  2   FROM Inventory
  3   WHERE Stock_Status = 'Out of Stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

3. Highest stocked products

```
SQL> SELECT p.Product_Name, i.Stock_Quantity
  2   FROM Product p
  3   JOIN Inventory i ON p.Product_ID = i.Product_ID
  4   WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);

PRODUCT_NAME
-----
----
STOCK_QUANTITY
-----
Sample Product
              4

Sample Product 2
              4
```

4. Average inventory quantity

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory
  2   FROM Inventory;

AVERAGE_INVENTORY
-----
3.5
```